

Proyecto 1	Python y Programación orientada a objetos
Docente	Jacqueline Aldridge Águila
Curso	Base de datos II
Fecha de entrega	Lunes 28 de septiembre, 12:00 (mediodía)
Ponderación	30% de la asignatura

Proyecto 1: Sistema de gestión bancaria

Objetivo y descripción del proyecto

Este proyecto tiene como objetivo implementar un sistema de gestión de cuentas bancarias utilizando Python y aplicando el paradigma de Programación Orientada a Objetos (POO). El sistema deberá permitir procesar, validar y gestionar información asociada a distintos tipos de cuentas bancarias, aplicando los conceptos y estructuras propios de este paradigma.

La tarea se divide en dos partes, que permitirán abordar progresivamente el procesamiento y la gestión de cuentas bancarias:

- En la primera parte se deberá implementar una función encargada de limpiar, validar y preparar los datos de las cuentas bancarias antes de ser utilizados por el sistema.
- En la segunda parte se deberá implementar un sistema de gestión de cuentas bancarias utilizando Programación Orientada a Objetos. Para ello, se deberán crear diferentes clases que representen las cuentas bancarias y el banco, definiendo sus respectivos atributos y métodos. En esta parte se gestionarán las distintas cuentas según sus características y operaciones y además se utilizarán los datos previamente procesados en la parte I de la tarea.

Se espera que el estudiante utilice correctamente los conceptos fundamentales del lenguaje de programación, así como los principios básicos de la programación orientada a objetos:

- funciones
- manejo de cadenas de caracteres
- listas y diccionarios
- manejo de excepciones
- clases y objetos
- atributos y métodos
- encapsulamiento
- herencia
- polimorfismo

Parte I: Función para el preprocesamiento de datos de cuentas bancarias (30 puntos)

Se debe implementar una función llamada ***preprocesar_cuentas***, cuyo objetivo será limpiar y validar los datos de las cuentas bancarias, retornando una lista de diccionarios con la información procesada.

Para ello:

- La función recibirá como parámetro una lista de diccionarios, donde cada diccionario representará una cuenta bancaria. Cada cuenta deberá contener las siguientes claves: ***titular***, ***numero_cuenta***, ***saldo_inicial*** y ***tipo*** (ahorro o corriente).
 - Dependiendo del tipo de cuenta, el diccionario podrá contener la clave ***tasa_interes*** (para cuentas de ahorro) o ***limite_giro*** (para cuentas corrientes).
- Los datos iniciales pueden venir con problemas, tales como:
 - campos obligatorios faltantes
 - espacios en blanco al comienzo o al final de los valores
 - tipos de datos incorrectos (por ejemplo ***saldo_inicial*** almacenado como texto)
 - montos negativos
 - números de cuenta duplicados.

La función deberá aplicar las siguientes reglas de limpieza y validación:

- ***titular***: Se deberán eliminar espacios en blanco al comienzo y al final del texto, y formatear el nombre utilizando mayúscula inicial en cada palabra.

Ejemplos:

" Ana Pérez " -> "Ana Pérez"

"JUAN CARLOS OYARZÚN" -> "Juan Carlos Oyarzún"

- ***numero_cuenta***: Debe corresponder a un número entero y único. Si existen números de cuenta duplicados, se deberá conservar únicamente el primer registro encontrado y descartar los siguientes registros. Si el valor recibido corresponde a una cadena de caracteres se deberán eliminar los espacios al comienzo y al final y convertirlo a número entero.
- ***saldo_inicial***: Debe corresponder a un número entero no negativo. Si el valor es una cadena de caracteres y no es posible convertir el valor a un número entero o si es negativo, la cuenta deberá descartarse.
- ***tipo***: Debe corresponder a uno de los siguientes valores: «ahorro» o «corriente». La validación no debe distinguir entre mayúsculas y minúsculas y se deberán ignorar los espacios en blanco al comienzo y al final. Si el valor no corresponde a ninguno de los dos tipos válidos, la cuenta deberá descartarse.
- ***tasa_interes***: Para las cuentas tipo «ahorro», si el campo no está presente o contiene un valor no válido (por ejemplo, una cadena de caracteres que no puede convertirse a número o un valor negativo), se deberá asignar el valor por defecto 0.5.
- ***limite_giro***: Para las cuentas tipo «corriente», si el campo no está presente o contiene un valor no válido (por ejemplo, una cadena de caracteres que no puede convertirse a número o un valor negativo), se deberá asignar el valor por defecto 100000.

- **titular y numero_cuenta:** Son campos obligatorios, por lo que toda cuenta a la que le falte alguno de estos campos deberá descartarse.
- Se deberán utilizar bloques **try/except** para capturar los errores que puedan producirse durante la conversión de tipos de datos (por ejemplo, al convertir cadenas de caracteres a números), evitando que el programa se detenga ante la presencia de datos inválidos.
- Se deberá implementar un script de prueba que:
 - Defina una lista de datos iniciales de ejemplo con al menos 20 registros, incluyendo tanto casos válidos como casos que presenten distintos tipos de errores, tales como campos faltantes, números de cuenta duplicados, montos inválidos o negativos, tipos de cuenta desconocidos, entre otros.
 - Ejecute la función **preprocesar_cuentas** utilizando la lista de datos de prueba.
 - Imprima por pantalla la lista final de cuentas limpias y validadas.

Parte II: Modelamiento de cuentas mediante programación orientada a objetos (70 puntos)

1. Implementar la clase **Cuenta**, que representará una cuenta bancaria genérica. Esta clase deberá contar con los siguientes atributos:

- **titular**: Nombre del titular de la cuenta.
- **numero_cuenta**: Identificador único de la cuenta.
- **saldo**: Monto de dinero disponible en la cuenta. Debe inicializarse en cero y declararse como un atributo privado.
- **tipo_cuenta**: Tipo de cuenta bancaria. Este atributo deberá ser establecido automáticamente por cada clase heredada.

Además, la clase **Cuenta** debe contar con los siguientes métodos:

- **obtener_informacion_basica(self)**: Debe retornar un mensaje que contenga, al menos, el titular y el número de cuenta.
- **consultar_saldo(self)**: Debe retornar el saldo actual de la cuenta.
- **depositar(self, monto)**: Debe aumentar el saldo en el monto indicado. El monto debe ser un número positivo.
- **girar(self, monto)**: Debe disminuir el saldo en el monto indicado, validando que exista saldo suficiente (cada subclase puede aplicar reglas adicionales).

2. Implementar la clase **Banco** con los siguientes atributos y métodos:

- **nombre**: Nombre de la entidad financiera.
- **cuentas**: Lista que almacenará objetos correspondientes a las distintas cuentas bancarias. (lista de objetos de la clase **Cuenta**)
- **abrir_cuenta(self, cuenta)**: Debe añadir el objeto cuenta a la lista de cuentas. Antes de realizar la operación, deberá verificar que no exista otra cuenta con el mismo número.
- **buscar_cuenta(self, numero_cuenta)**: Debe buscar una cuenta por su número y retornar la cuenta encontrada. Si no existe, deberá indicar que la cuenta no fue encontrada.
- **transferir(self, numero_origen, numero_destino, monto)**: Debe buscar ambas cuentas en el banco, girar el monto desde la cuenta de origen y depositarlo en la cuenta de destino. Se deberá validar que ambas cuentas existan y que el monto sea válido.
- **mostrar_cuentas(self)**: Debe mostrar por pantalla todas las cuentas registradas en el banco, ordenadas de menor a mayor según el número de cuenta.

Los montos depositados, girados o transferidos, así como el saldo de las cuentas, no podrán ser modificados accediendo directamente a los atributos. Estas operaciones deberán realizarse obligatoriamente mediante los métodos definidos para tal propósito.

3. Implementar las siguientes clases heredadas de **Cuenta**, correspondientes a los distintos tipos de cuentas bancarias:
- **CuentaDeAhorro** con los siguientes atributos y métodos:
 - **tasa_interes**: Porcentaje de interés mensual que se aplica sobre el saldo de la cuenta.
 - **aplicar_interes(self)**: Debe aumentar el saldo aplicando la tasa de interés correspondiente sobre el saldo actual.
 - **CuentaCorriente** con los siguientes atributos y métodos:
 - **limite_giro**: Monto máximo de sobregiro permitido. Debe corresponder a un número positivo.
 - **girar(self, monto)**: Debe permitir realizar giros aunque el saldo resulte negativo, siempre que este no supere el límite de sobregiro establecido por **limite_giro**.

Cada una de estas clases debe tener su propio método **obtener_informacion_basica()**. Este método debe ser redefinido en cada clase heredada, mostrando información específica según el tipo de cuenta.

Cada clase heredada deberá establecer automáticamente el atributo **tipo_cuenta** correspondiente («ahorro» o «corriente»).

Consideraciones para la implementación

- La Cuenta de ahorro no permitirá girar montos que dejen el saldo bajo cero (no admite sobregiro).
- Los montos depositados, girados o transferidos deben ser siempre números positivos.
- Los atributos privados (número de cuenta y saldo) deben ser manipulados y consultados únicamente a través de métodos definidos en la clase, respetando el principio de encapsulamiento.
- Se deben utilizar excepciones para manejar errores en caso de que falte información o los datos proporcionados no sean válidos.
- Si es necesario puedes implementar métodos o atributos extra que faciliten la implementación del código.
- Las clases heredadas de **Cuenta** deben automáticamente establecer el atributo **tipo_cuenta**.
- Se debe implementar un script que simule la apertura de cuentas y la realización de operaciones (depósitos, giros, transferencias), de forma que se utilicen todas las clases y métodos implementados.
- Se puede reutilizar la función de preprocesamiento de datos del ejercicio anterior para generar la lista de cuentas iniciales del script de simulación.

Flujo de trabajo esperado

Se espera que el script de simulación siga el siguiente flujo:

1. Generar la lista de diccionarios con los datos iniciales de las cuentas.
2. Procesar la lista utilizando la función implementada.
3. Crear un objeto Banco (sin cuentas registradas).
4. A partir de los datos ya procesados, crear los objetos de cuenta correspondientes y abrir las cuentas en el banco mediante el método específico para esto.
5. Aplicar sobre estas cuentas los distintos métodos implementados (depósitos, giros, transferencias, consulta de saldo, aplicación de interés, etc.).

Declaración de IA

Cada estudiante debe incluir en su repositorio un archivo con una declaración de uso de IA. En ella debe indicar si utilizó o no herramientas de inteligencia artificial durante el desarrollo del proyecto, y en caso afirmativo, especificar cuáles y para qué las usó: si fue para resolver dudas conceptuales, para depurar errores, para revisar o mejorar código ya escrito, para generar los datos de prueba, o para cualquier otro fin.

Si parte del código entregado fue generado directamente por una IA, esto debe quedar explícitamente señalado en la declaración, indicando qué función o sección corresponde y cómo fue adaptada o comprendida por el estudiante.

El uso de herramientas de inteligencia artificial (IA) está permitido en el apoyo del desarrollo de la tarea y en la resolución de errores, pero no se permite la entrega de tareas totalmente generadas con IA.

Evaluación

Criterio	Puntaje
Limpieza y formato de datos	8 puntos
Validaciones	12 puntos
Manejo de excepciones	5 puntos
Atributos y encapsulamiento	15 puntos
Métodos	30 puntos
Herencia y polimorfismo	15 puntos
Scripts de simulación	15 puntos

Consideraciones:

- La tarea es individual.
- Para que un proyecto sea evaluado debes subir el código a un repositorio de GitHub, en caso de que el repositorio sea privado, puedes dar acceso al usuario JacquelineAldridge. Se evaluará el último commit antes de la fecha de entrega.
- Se evaluarán con nota mínima aquellos proyectos que no incluyan declaración de IA.
- No se aceptarán trabajos con una metodología diferente a la vista en clases.
- Además de *try/except*, pueden usar *raise* para lanzar excepciones cuando un método detecte una condición inválida.
- El código entregado debe ser original y reflejar el entendimiento del estudiante. Será motivo de evaluación con nota mínima (1.0) en los siguientes casos:
 - Una alta similitud entre dos o más tareas.
 - Tareas en las cuales se detecte entrega de código generado íntegramente por una herramienta de IA sin evidencias de adaptación.

En ambos casos, el estudiante podrá apelar esta evaluación mediante una interrogación oral, en la cual deberá demostrar dominio sobre el trabajo entregado.